

Student Management System — MVC Solution

Stack: Spring Boot + Hibernate (JPA) + MySQL/H2 + Thymeleaf

Step 1 – Spring Boot Dependencies (pom.xml)

Add these dependencies when creating the project via Spring Initializr.

```
<!-- Spring Web -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- Thymeleaf -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<!-- Spring Data JPA -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<!-- MySQL Driver -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

Step 2 – Database Configuration (application.properties)

Located at: `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:mysql://localhost:3306/mvc_lab
spring.datasource.username=root
spring.datasource.password=yourpassword

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Step 3 – Model / Entity Layer (Student.java)

Package: `com.example.demo.model` | Hibernate entity — maps to the student table.

```
package com.example.demo.model;

import jakarta.persistence.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String course;

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
```

```

    public void setName(String name)    { this.name = name; }

    public String getEmail()           { return email; }
    public void setEmail(String email)  { this.email = email; }

    public String getCourse()           { return course; }
    public void setCourse(String course) { this.course = course; }
}

```

Step 4 – Repository Layer (StudentRepository.java)

Package: com.example.demo.repository | No implementation needed — JPA handles CRUD automatically.

```

package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.demo.model.Student;

public interface StudentRepository extends JpaRepository<Student, Long> {
}

```

Step 5 – Service Layer (StudentService.java)

Package: com.example.demo.service | Business logic layer — maintains separation of concerns.

```

package com.example.demo.service;

import org.springframework.stereotype.Service;
import java.util.List;
import com.example.demo.model.Student;
import com.example.demo.repository.StudentRepository;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    public void saveStudent(Student student) {
        repository.save(student);
    }
}

```

Step 6 – Controller Layer (StudentController.java)

Package: com.example.demo.controller | Handles HTTP requests and delegates to the service layer.

```

package com.example.demo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import com.example.demo.model.Student;
import com.example.demo.service.StudentService;

@Controller
public class StudentController {

    private final StudentService service;
}

```

```

    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping("/")
    public String viewHomePage(Model model) {
        model.addAttribute("students", service.getAllStudents());
        model.addAttribute("student", new Student());
        return "index";
    }

    @PostMapping("/save")
    public String saveStudent(@ModelAttribute Student student) {
        service.saveStudent(student);
        return "redirect:/";
    }
}

```

Step 7 – View / Thymeleaf Template (index.html)

Path: *src/main/resources/templates/index.html* | Registration form + student list table.

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Student Management</title>
</head>
<body>

<h2>Add Student</h2>
<form th:action="@{/save}" th:object="${student}" method="post">
    Name: <input type="text" th:field="**{name}" /><br>
    Email: <input type="text" th:field="**{email}" /><br>
    Course: <input type="text" th:field="**{course}" /><br>
    <button type="submit">Save</button>
</form>

<h2>Student List</h2>
<table border="1">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Course</th>
    </tr>
    <tr th:each="s : ${students}">
        <td th:text="${s.id}"></td>
        <td th:text="${s.name}"></td>
        <td th:text="${s.email}"></td>
        <td th:text="${s.course}"></td>
    </tr>
</table>

</body>
</html>

```